Blue X V1 Role Document

Backend Developer

This document explains the backend responsibility for Version 1 of the Blue X payment verification app.

The backend is the source of truth. It handles authentication, validation, payment verification, provider API calls, saving data, duplicate detection, dashboard summaries, and security rules.

Main Goal

Build a Django REST backend that allows the mobile app to:

- Register and login users
- Load/update profile and business details
- Submit payment verification requests
- Receive clean verification results
- Save transaction history
- Load dashboard summaries and transaction history

Backend Structure

Backend structure for V1:

```
backend/
  accounts/
    models.py
    serializers.py
    views.py
    urls.py
    permissions.py

  verification/
    models.py
    serializers.py
    views.py
    urls.py
    services.py
    providers/
```

```
      base.py
      qbirr.py
      odit.py
      verify_et.py

  analytics/
    services.py
    serializers.py
    views.py
    urls.py
```

It is okay to add more files if needed, but keep the project small and readable.

Accounts App

The `accounts` app handles user account and business profile.

Models:

- User or CustomUser
- BusinessProfile

User fields:

- `id`
- `full_name`
- `phone_number`
- `email`
- `password`
- `is_active`
- `created_at`
- `updated_at`

BusinessProfile fields:

- `id`
- `user`
- `business_name`
- `business_type`
- `receiver_name`
- `receiver_account`
- `default_provider`
- `created_at`

- `updated_at`

Serializers:

- RegisterSerializer
- LoginSerializer
- UserSerializer
- BusinessProfileSerializer
- ProfileSerializer

Views:

- RegisterView
- LoginView
- LogoutView
- ProfileView

APIs:

```
POST /api/accounts/register/
POST /api/accounts/login/
POST /api/accounts/logout/
GET /api/accounts/profile/
PUT /api/accounts/profile/
```

Rules:

- Passwords must be hashed.
- Do not save plain text passwords.
- User must be logged in to access profile.
- User can only access their own profile.
- V1 can skip phone number OTP verification.
- Phone number format should still be validated.

Verification App

The `verification` app handles the core payment verification loop.

Models:

- VerificationTransaction
- ScanUsage

VerificationTransaction fields:

- `id`
- `user`
- `business_profile`
- `provider`
- `bank_name`
- `receipt_id`
- `transaction_reference`
- `receipt_url`
- `sender_name`
- `sender_phone`
- `receiver_name`
- `receiver_account`
- `amount`
- `currency`
- `status`
- `verified_at`
- `provider_transaction_date`
- `raw_provider_response`
- `failure_reason`
- `is_duplicate`
- `created_at`
- `updated_at`

Allowed statuses:

- `verified`
- `pending`
- `failed`
- `unknown`

ScanUsage fields:

- `id`
- `user`
- `month`
- `scan_count`
- `free_limit`
- `created_at`

- `updated_at`

Serializers:

- VerificationRequestSerializer
- VerificationTransactionSerializer
- TransactionHistorySerializer
- DashboardSummarySerializer

Views:

- VerifyPaymentView
- TransactionHistoryView
- DashboardSummaryView

Services:

- Provider selection service
- Provider API call service
- Response normalization service
- Save verification transaction service
- Duplicate reference service
- Scan usage service

Verification Flow

The backend verification endpoint should follow this order:

```
Receive request from mobile app
-> Confirm user is authenticated
-> Validate request serializer
-> Check scan usage limit
-> Check duplicate receipt/reference
-> Select provider service
-> Call provider API or mock provider
-> Normalize provider response
-> Save VerificationTransaction
-> Increase ScanUsage
-> Return clean response to frontend
```

The mobile app should receive a clean response only. The raw provider response should be stored for backend/debugging but should not be casually exposed.

Provider Services

The provider service layer should make all providers look the same to the rest of our backend.

Provider services can include:

- qbirr
- v.odit.et
- verify.et
- mock provider for development

Every provider service should return:

```
{
  "status": "verified",
  "amount": "500.00",
  "currency": "ETB",
  "sender_name": "Customer Name",
  "receiver_name": "Natnael Example",
  "provider": "cbe",
  "transaction_date": "2026-06-24T14:20:00Z",
  "reference": "FT23001234ABC",
  "message": "Payment verified successfully",
  "raw_response": {}
}
```

If real provider API access is not ready, use a mock provider first so frontend and backend integration can continue.

Dashboard / Analytics

Dashboard data should be calculated from saved `VerificationTransaction` records.

Dashboard API should calculate:

- Total verified amount
- Total scans
- Verified count
- Pending count
- Failed count
- Unknown count
- Breakdown by provider/bank
- Recent transactions
- Weekly summary

- Monthly summary
- Yearly summary

Every dashboard query must filter by logged-in user.

Security Rules

- Scanner API requires login.
- Dashboard API requires login.
- Profile API requires login.
- Users can only see their own data.
- Backend must validate all frontend input.
- Backend must not trust frontend status.
- External API keys must stay in backend environment variables.
- Duplicate receipt/reference must be handled.
- Raw provider response should not be shown unnecessarily.

Done When

This role is complete for V1 when:

- Register/login/profile APIs work.
- Verification endpoint works with real or mock provider.
- Verification result is saved to database.
- Scan usage increases after verification.
- Duplicate references are detected.
- Dashboard summary API returns correct user data.
- Transaction history API returns correct user data.
- Protected APIs reject unauthenticated requests.
- User data is isolated between users.